

Implementando Padrões de Teste (Test Patterns)

1. Introdução e Contexto

No trabalho anterior, focamos em detectar e refatorar Test Smells. Vimos que código de teste mal cheiroso, como testes com *Setup Obscuro* (Obscure Setup) ou *Testes Frágeis* (Fragile Tests) acoplados a dependências externas, é difícil de ler, manter e, o mais importante, é ineficaz para garantir a qualidade.

Agora, vamos avançar da *correção* (Refatoração) para a *prevenção*. Em vez de apenas limpar a sujeira, vamos aprender a construir testes limpos desde o início.

Neste trabalho, vocês atuarão como desenvolvedores de software sênior, cuja tarefa é construir uma suíte de testes robusta para um novo serviço de e-commerce. Para isso, vocês aplicarão Padrões de Teste essenciais para resolver dois problemas centrais:

1. **Criação de Dados de Teste:** Como criar objetos complexos (usuários, carrinhos de compra) de forma legível e flexível? (Padrões: Object Mother, Data Builder)
2. **Isolamento de Dependências:** Como testar nossa lógica de negócio sem depender de serviços externos reais, como gateways de pagamento ou envio de e-mails? (Padrões: Stubs, Mocks)

2. Objetivos de Aprendizagem

Ao final deste trabalho, o aluno deverá ser capaz de:

- Implementar o padrão **Object Mother** para criar instâncias de objetos simples e fixos.
- Implementar o padrão **Data Builder** para criar objetos complexos de forma fluente e customizável.
- Diferenciar conceitualmente e na prática o uso de **Stubs** e **Mocks**.
- Aplicar **Stubs** para fornecer respostas pré-definidas e focar na Verificação de Estado (State Verification).
- Aplicar **Mocks** para validar interações e focar na Verificação de Comportamento (Behavior Verification).
- Escrever testes de unidade limpos e isolados, seguindo o padrão **Arrange, Act, Assert (AAA)**.

3. O Projeto Base

Para este trabalho, utilizaremos um projeto que simula um serviço de Checkout de E-commerce. O código-fonte do sistema (SUT - System Under Test) já está implementado, mas falta a suíte de testes.

- **Repositório do Projeto:** <https://github.com/CleitonSilvaT/test-pattern>

O repositório contém:

- Um diretório `src/domain/` com as classes de negócio (ex: `User.js`, `Carrinho.js`, `Item.js`, `Pedido.js`).
- Um diretório `src/services/` com as dependências externas (ex: `GatewayPagamento.js`, `EmailService.js`, `PedidoRepository.js`).
- O arquivo `src/services/CheckoutService.js` (o SUT que vocês deverão testar).
- Um arquivo `package.json` com as dependências básicas (Jest).
- Um diretório `__tests__/` vazio, onde seus testes serão criados.

4. Etapas do Trabalho

Etapa 1: Preparação do Ambiente

1. Faça um **fork** do repositório para sua conta do GitHub.
2. Clone o seu fork para sua máquina local.
3. Instale as dependências do projeto com o comando: `npm install`.
4. Crie seu arquivo de teste: `__tests__/CheckoutService.test.js`.

Etapa 2: Padrão Object Mother (Setup Fixo)

1. No seu diretório de testes, crie uma pasta `builders` (ex: `__tests__/builders/`).
2. Dentro dela, crie o arquivo `UserMother.js`.
3. Implemente métodos estáticos para criar usuários comuns, como `UserMother.umUsuarioPadrao()` e `UserMother.umUsuarioPremium()`.
4. **Justificativa:** Este padrão é útil para entidades que são simples e raramente mudam entre os testes (como tipos de usuário).

Etapa 3: Padrão Data Builder (Setup Flexível)

1. O Carrinho é um objeto complexo: pode ter diferentes usuários, vários itens, ou estar vazio. Um *Object Mother* aqui levaria a uma explosão de métodos.
2. Crie o arquivo `__tests__/builders/CarrinhoBuilder.js`.
3. Implemente o padrão *Builder*, com:
 - Um construtor com valores padrão (ex: um carrinho com 1 item padrão).
 - Métodos fluentes para customização (ex: `.comUser(user)`, `.comItens([...])`, `.vazio()`).
 - Um método `.build()` que retorna a instância final do Carrinho.
4. **Justificativa:** Este padrão resolve o *Test Smell* de "Setup Obscuro", tornando explícito

apenas o que é importante para aquele cenário de teste.

Etapa 4: Padrão Stub (Verificação de Estado)

1. Seu primeiro teste será validar o cenário de falha no pagamento.
2. Escreva um teste `describe('quando o pagamento falha, ...')`
3. **Arrange:**
 - Use o `CarrinhoBuilder` para criar um carrinho.
 - Crie um **Stub** para o `GatewayPagamento`. A única função dele é retornar `{ success: false }` quando `cobrar()` for chamado.
 - Use `jest.fn()`: `const gatewayStub = { cobrar: jest.fn().mockResolvedValue({ success: false }) };`
 - Crie "dublês vazios" (Dummies) para as outras dependências (`Repository` e `EmailService`), pois elas não devem ser chamadas.
4. **Act:** Chame o `checkoutService.processarPedido(...)`.
5. **Assert (Verificação de Estado):** Verifique se o *resultado* (o estado) do método é `null`, como esperado.
 - `expect(pedido).toBeNull();`

Etapa 5: Padrão Mock (Verificação de Comportamento)

1. Agora, teste o cenário de sucesso para um cliente Premium (que deve ter desconto).
2. Escreva um teste `describe('quando um cliente Premium finaliza a compra, ...')`
3. **Arrange:**
 - Use o `UserMother` para pegar um usuário "PREMIUM".
 - Use o `CarrinhoBuilder` para criar um carrinho com esse usuário e R\$ 200,00 em itens.
 - Crie **Stubs** para o `GatewayPagamento` (retornando `{ success: true }`) e para o `PedidoRepository` (retornando um pedido salvo).
 - Crie um **Mock** para o `EmailService`, pois queremos verificar se ele foi chamado, e *com quais argumentos*.
4. **Act:** Chame o `checkoutService.processarPedido(...)`.
5. **Assert (Verificação de Comportamento):**
 - Verifique se o `GatewayPagamento` foi chamado com o valor correto (R\$ 180,00, aplicando o desconto de 10%).
 - `expect(gatewayStub.cobrar).toHaveBeenCalledWith(180, ...);`
 - Verifique se o `EmailService` foi chamado 1 vez, com o e-mail e a mensagem corretos.
 - `expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);`
 - `expect(emailMock.enviarEmail).toHaveBeenCalledWith('premium@email.com', 'Seu Pedido foi Aprovado!', ...);`

Etapa 6: Validação Final

1. Execute a suíte de testes completa (`npm test`). Todos os seus novos testes devem passar.
2. Faça o commit e o push das suas alterações, incluindo os novos arquivos de teste e os builders, para o seu repositório no GitHub.

5. O Que Entregar

1. **Link do Repositório GitHub:** O link para o seu fork do projeto, contendo o arquivo `__tests__/CheckoutService.test.js` e a pasta `__tests__/builders/` com os padrões implementados.
2. **Relatório Escrito (PDF, 2-4 páginas):** Um documento contendo:
 - **Capa:** Nome da disciplina, nome do trabalho, seu nome completo e matrícula.
 - **Padrões de Criação de Dados (Builders):**
 - Explique por que o `CarrinhoBuilder` foi usado em vez de um `CarrinhoMother`.
 - Mostre um exemplo de teste "Antes" (como seria um setup manual complexo) e o "Depois" (o setup usando seu `Data Builder`).
 - Justifique como o `Builder` melhora a legibilidade e manutenção do teste.
 - **Padrões de Test Doubles (Mocks vs. Stubs):**
 - Escolha o seu teste de "sucesso Premium" (Etapa 5).
 - Nele, identifique qual dependência foi usada como **Stub** e qual foi usada como **Mock**.
 - Explique *por que* o `GatewayPagamento` foi (principalmente) um `Stub`, e *por que* o `EmailService` foi um `Mock`. (Dica: pense sobre "Verificação de Estado vs. Comportamento").
 - **Conclusão:** Uma breve reflexão sobre como o uso deliberado de Padrões de Teste (`Builders` e `Doubles`) ajuda a prevenir *Test Smells* e contribui para uma suíte de testes sustentável.

6. Critérios de Avaliação

- **Implementação de Padrões de Criação (30%):** Os padrões `Object Mother` e `Data Builder` foram implementados corretamente, com o `Builder` oferecendo uma API fluente.
- **Implementação de Test Doubles (40%):** O uso de `jest.fn()` (ou `mocks` similares) está correto, demonstrando a diferença clara entre `Stubs` (para controlar o fluxo/retorno) e `Mocks` (para verificar interações).
- **Qualidade dos Testes (10%):** Os testes seguem o padrão AAA, são focados e cobrem os cenários propostos (falha, desconto, envio de e-mail).
- **Qualidade do Relatório Escrito (20%):** Clareza nas explicações, profundidade na análise (`Builder` vs `Mother`, `Stub` vs `Mock`) e cumprimento de todos os itens solicitados.

7. Recursos Adicionais

- **Documentação do Jest (Mock Functions):** <https://jestjs.io/docs/mock-functions>
- **Artigo de Referência (Martin Fowler - Mocks Aren't Stubs):** <https://martinfowler.com/articles/mocksArentStubs.html>
- **Livro (Meszaros, Gerard - xUnit Test Patterns):** <http://xunitpatterns.com/>